MINI PROJECT REPORT

RTL TO GDSII USING SYNOPSYS TOOLS

Physical Design of 8-bit PWM

SUBMITTED BY,

ANVAY PAWAR

INTRODUCTION

In modern digital system design, efficient signal generation and power control techniques are essential for embedded and real-time applications. Pulse Width Modulation (PWM) is a widely used digital technique for controlling the average power delivered to a load by varying the duty cycle of a periodic signal while maintaining a constant frequency. PWM is commonly used in motor control, power converters, LED dimming, and communication systems. From a VLSI perspective, PWM involves synchronous sequential logic and comparator-based decision circuitry, making it suitable for studying ASIC design and timing analysis.

This project focuses on the design, verification, synthesis, and physical implementation of an 8-bit PWM module using Verilog HDL. The architecture consists of an 8-bit synchronous counter and a comparator, where the duty cycle input determines the pulse width of the output signal. The complete RTL-to-GDSII flow is implemented using Synopsys tools, including VCS for simulation, Design Compiler for synthesis, IC Compiler II for physical design, and PrimeTime for static timing analysis. The objective is to achieve a functionally correct and timing-closed design following standard ASIC design methodology.

The implementation is carried out using the SAED32nm technology library, which provides standard cells and timing models for advanced-node ASIC design. The 32nm technology enables higher integration density and improved performance, allowing realistic evaluation of timing, area, and design constraints within an industrial-style design flow.

OBJECTIVES

The primary objective of this project is to design, verify, and physically implement an 8-bit Pulse Width Modulation (PWM) module using standard ASIC design methodologies. The design flow spans from RTL development to GDS-II generation, with emphasis on functional correctness, synchronous operation, and timing closure. This project provides hands-on exposure to the Synopsys EDA toolchain and a clear understanding of the major stages involved in frontend and backend VLSI implementation.

Specific Objectives:

1. To design and develop RTL code for an 8-bit PWM module consisting of a synchronous counter and comparator logic using Verilog HDL.
2. To ensure fully synchronous operation of the PWM design with proper clock and reset handling.
3. To verify the functional correctness of the PWM module through simulation using Synopsys VCS and waveform analysis.
4. To perform logic synthesis using Design Compiler, optimizing the design for area, power, and timing constraints.
5. To execute the physical design flow using IC Compiler II, including floorplanning, pin placement, power planning, placement, clock tree synthesis (CTS), and routing.
6. To perform Static Timing Analysis (STA) using PrimeTime and ensure that the design meets setup and hold timing requirements with positive slack.
7. To gain practical experience in the complete RTL-to-GDSII flow using SAED32nm technology, reinforcing knowledge of modern ASIC design methodology.

RTL CODE AND SIMULATION

RTL CODE

The RTL design of the 8-bit PWM module consists of a synchronous 8-bit counter and comparator logic. The counter increments on each rising edge of the clock and resets based on the reset signal. The comparator compares the counter value with the 8-bit duty cycle input to generate the PWM output.

When the counter value is less than the duty cycle input, the output is high; otherwise, it is low. This structure produces a pulse-width modulated signal with a duty cycle proportional to the input value.

```
`timescale 1ns/1ps   // <--- Specify timescale

// 8-bit PWM Generator

module pwm_8bit (

   input wire clk,            // System clock

   input wire rst,            // Active-high reset

   input wire [7:0] duty,      // 8-bit duty cycle input (0-255)

   output reg pwm_out          // PWM output

);


   // 8-bit counter

   reg [7:0] counter;


   // Counter logic

   always @(posedge clk or posedge rst) begin
```

```
    if (rst)
        counter <= 8'b0;
    else
        counter <= counter + 1'b1;  // Increment counter every clock
  end

  // PWM output logic
  always @(posedge clk or posedge rst) begin
    if (rst)
        pwm_out <= 1'b0;
    else
        pwm_out <= (counter < duty) ? 1'b1 : 1'b0;
  end

endmodule
```

TESTBENCH:-

```
`timescale 1ns/1ps

module tb_pwm_8bit;

  reg clk;
  reg rst;
```

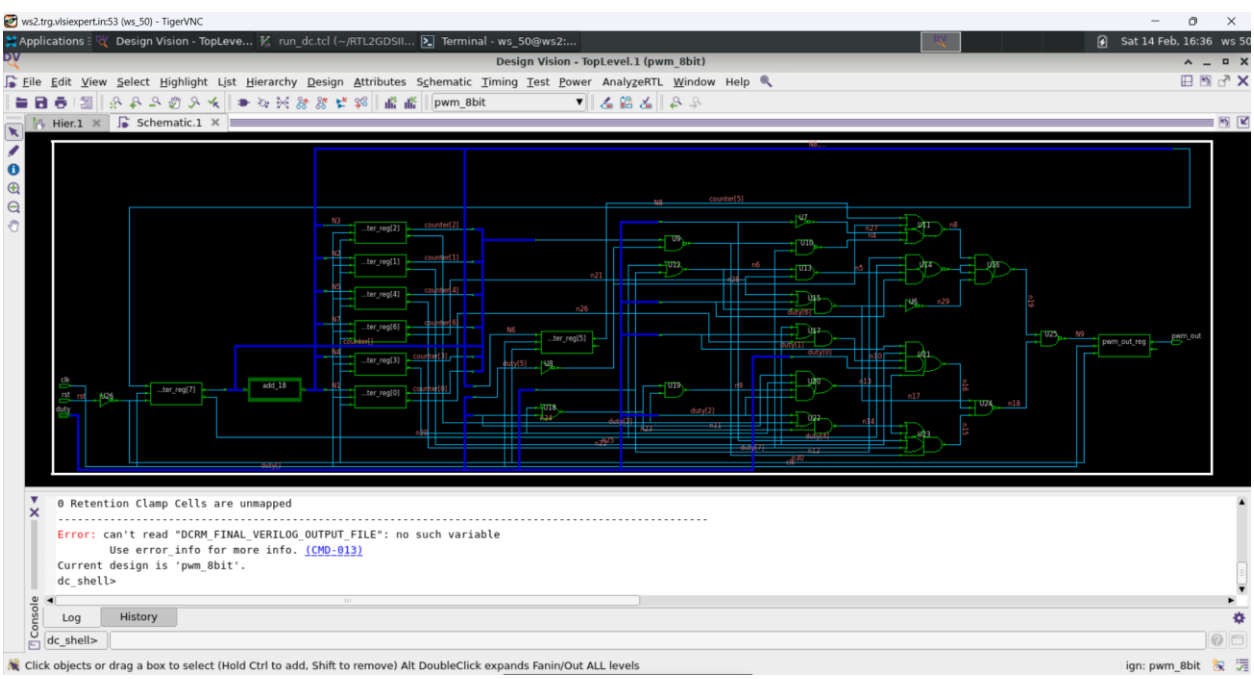
```
reg [7:0] duty;
wire pwm_out;

pwm_8bit uut (
    .clk(clk),
    .rst(rst),
    .duty(duty),
    .pwm_out(pwm_out)
);

// VCD Dump
initial begin
    $dumpfile("pwm.vcd");
    $dumpvars(0, tb_pwm_8bit);
end

initial begin
    $fsdbDumpfile("pwm.fsdb");
    $fsdbDumpvars(0, tb_pwm_8bit);
end

// Clock generation (100 MHz)
initial clk = 0;
```

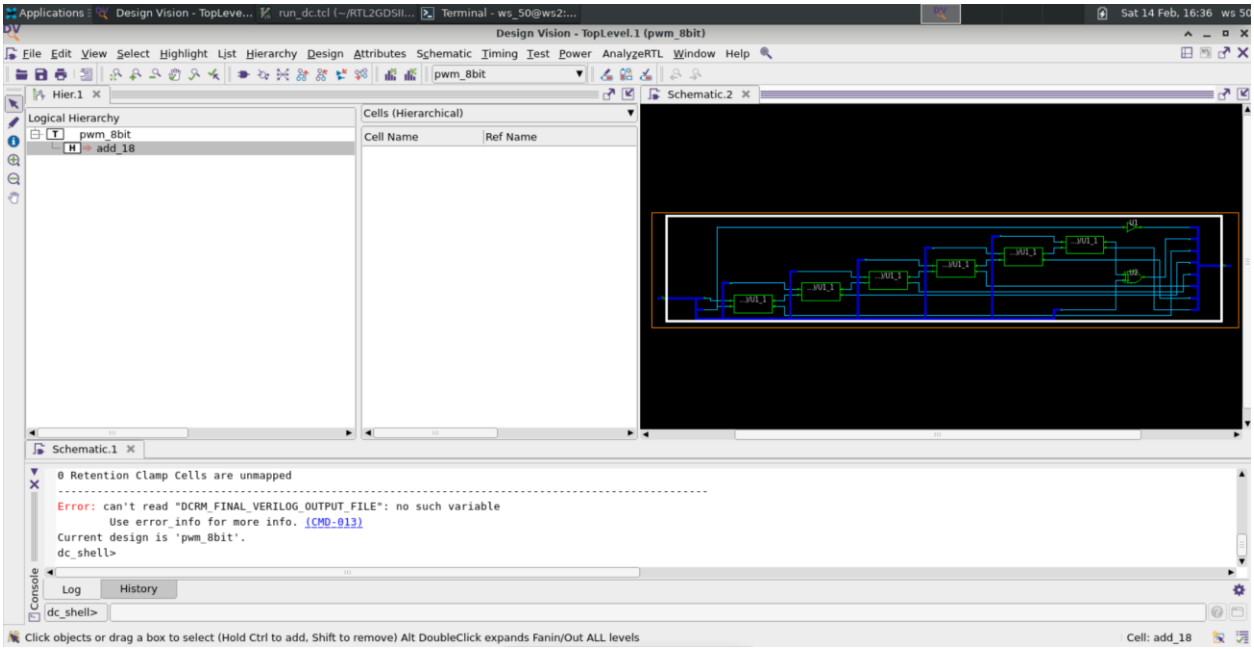
```
    always #5 clk = ~clk;

    // Test stimulus
    initial begin
        rst = 1;
        duty = 8'd0;

        #20 rst = 0;

        #100 duty = 8'd64;   // 25%
        #200 duty = 8'd128;  // 50%
        #200 duty = 8'd192;  // 75%
        #200 duty = 8'd255;  // 100%

        #200 $finish;
    end

endmodule
```

VCS Simulation Report :-

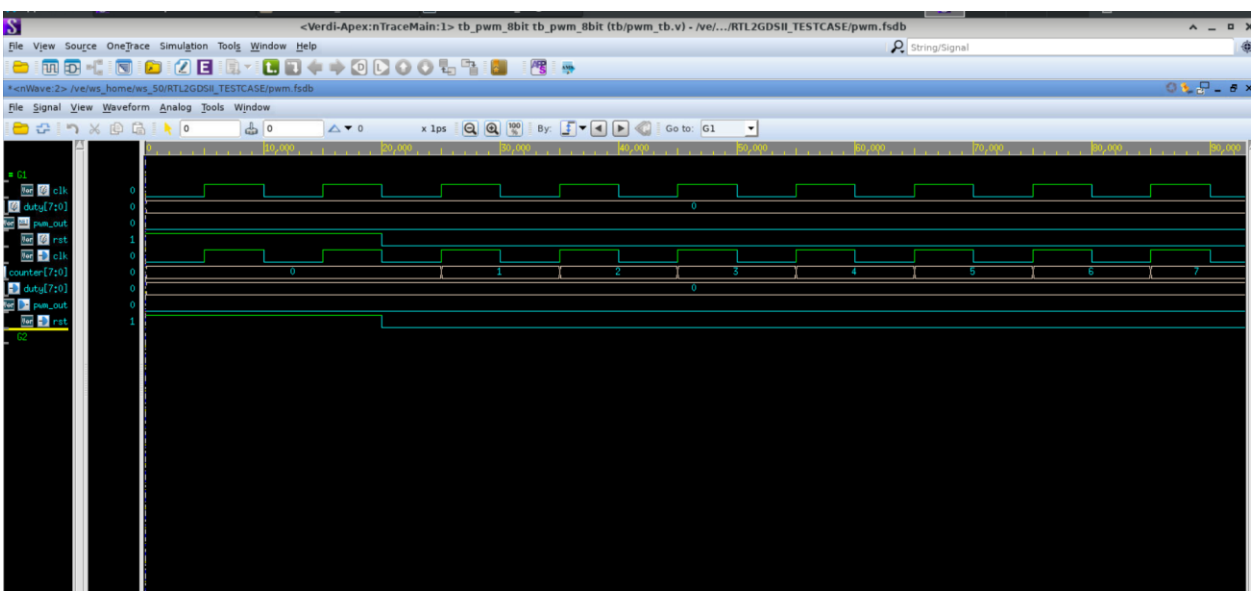
```
Terminal - ws_50@ws2:~/RTL2GDSII_TESTCASE
File  Edit  View  Terminal  Tabs  Help
[ws_50@ws2 RTL2GDSII_TESTCASE]$ ./simv
Info: [VCS_SAVE_RESTORE_INFO] ASLR (Address Space Layout Randomization) is detec
ted on the machine. To enable $save functionality, ASLR will be switched off and
 simv re-executed.
Please use '-no_save' simv switch to avoid re-execution or '-suppress=ASLR_DETEC
TED_INFO' to suppress this message.
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP2-7_Full64; Runtime version U-2023.03-SP2-7_Full64;
  Feb 14 16:24 2026
*Verdi* Loading libsscore_vcs202303.so
FSDB Dumper for VCS, Release Verdi_U-2023.03-SP2-7, Linux x86_64/64bit, 06/23/20
24
(C) 1996 - 2024 by Synopsys, Inc.
*Verdi* : Create FSDB file 'pwm.fsdb'
*Verdi* : Begin traversing the scope (tb_pwm_8bit), layer (0).
*Verdi* : End of traversing.
$finish called from file "tb/pwm_tb.v", line 44.
$finish at simulation time              920000
           V C S   S i m u l a t i o n   R e p o r t
Time: 920000 ps
CPU Time:      0.150 seconds;       Data structure size:   0.0Mb
Sat Feb 14 16:24:44 2026
[ws_50@ws2 RTL2GDSII_TESTCASE]$
```

RTL Schematics





Output waveform



LOGIC SYNTHESIS USING DC SHELL

RTL synthesis is the process of converting the Verilog-based RTL description of the 8-bit PWM module into a gate-level netlist that can be physically implemented on an ASIC. During this stage, the high-level behavioral description of the counter and comparator logic is mapped to technology-specific standard cells from the SAED32nm library.

Using Synopsys Design Compiler (DC Shell), the design is analyzed, elaborated, and optimized based on the provided timing constraints (SDC). The synthesis process generates a mapped netlist composed of logic gates and flip-flops while optimizing for area, power, and timing. The resulting gate-level netlist serves as the input for the subsequent physical design stages in IC Compiler II.

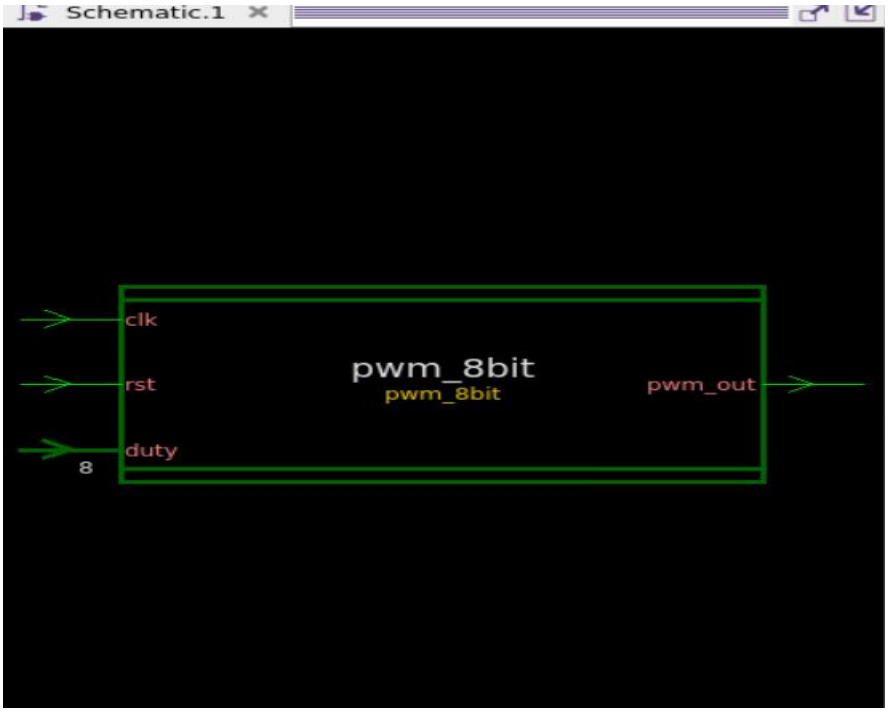
Dc_run.tcl file script

```
source -echo -verbose ./rm_setup/dc_setup.tcl

set DESIGN_NAME pwm_8bit

set DCRM_FINAL_VERILOG_OUTPUT_FILE pwm.mapped.v

set RTL_SOURCE_FILES ./../rtl/pwm.v


define_design_lib WORK -path ./WORK


#set_dont_use [get_lib_cells */FADD*]

#set_dont_use [get_lib_cells */HADD*]

#set_dont_use [get_lib_cells */AO*]

#set_dont_use [get_lib_cells */OA*]

#set_dont_use [get_lib_cells */NAND*]

#set_dont_use [get_lib_cells */XOR*]
```

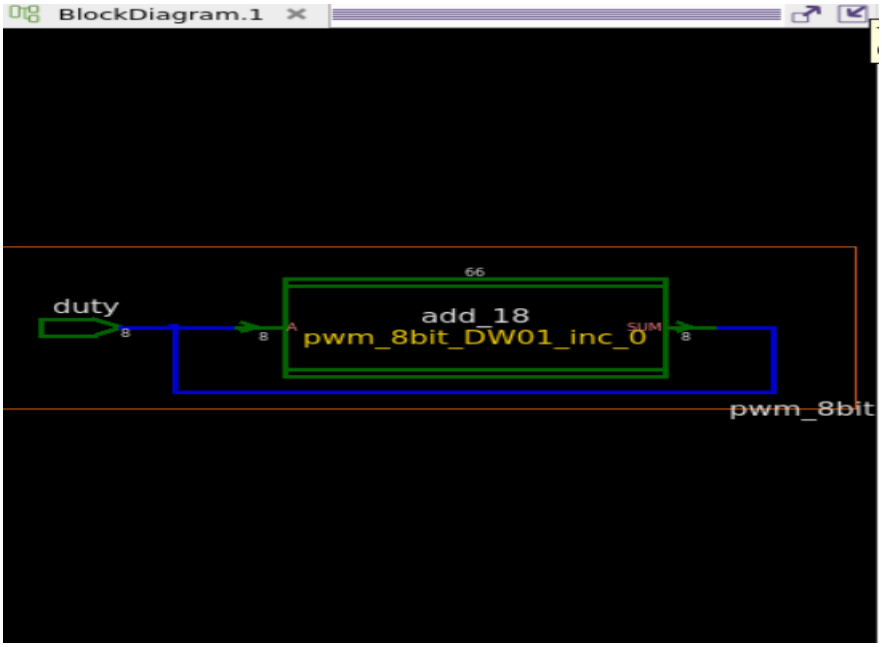
```
#set_dont_use [get_lib_cells */NOR*]
#set_dont_use [get_lib_cells */XNOR*]
#set_dont_use [get_lib_cells */MUX*]

analyze -format verilog ${RTL_SOURCE_FILES}
elaborate ${DESIGN_NAME}
current_design



read_sdc ./../CONSTRAINTS/pwm.sdc

compile

#compile_ultra
#report_timing
write -format verilog -hierarchy -output
${RESULTS_DIR}/${DCRM_FINAL_VERILOG_OUTPUT_FILE}
```

\

Schematics



Block diagram:-



Report QOR :-

```
****************************************
Report : qor
Design : pwm_8bit
Version: V-2023.12-SP4
Date   : Sat Feb 14 16:37:28 2026
****************************************


  Timing Path Group 'clk'
  -----------------------------------
  Levels of Logic:               6.00
  Critical Path Length:          0.58
  Critical Path Slack:           7.13
  Critical Path Clk Period:     10.00
  Total Negative Slack:          0.00
  No. of Violating Paths:        0.00
  Worst Hold Violation:          0.00
  Total Hold Violation:          0.00
  No. of Hold Violations:        0.00
  -----------------------------------


  Cell Count
  -----------------------------------
  Hierarchical Cell Count:          1
  Hierarchical Port Count:         16
  Leaf Cell Count:                 38
  Buf/Inv Cell Count:               5
  Buf Cell Count:                   0
  Inv Cell Count:                   5
  CT Buf/Inv Cell Count:            0
  Combinational Cell Count:        29
  Sequential Cell Count:            9
  Macro Count:                      0
  -----------------------------------
```

```
Area
-----------------------------------
Combinational Area:       69.635457
Noncombinational Area:    64.044289
Buf/Inv Area:              6.607744
Total Buffer Area:             0.00
Total Inverter Area:           6.61
Macro/Black Box Area:      0.000000
Net Area:                  9.359675
-----------------------------------
Cell Area:               133.679746
Design Area:             143.039421


Design Rules
-----------------------------------
Total Number of Nets:            61
Nets With Violations:             0
Max Trans Violations:             0
Max Cap Violations:               0
-----------------------------------


Hostname: ws2.trg.vlsiexpert.in

Compile CPU Statistics
----------------------------------------
Resource Sharing:                    0.04
Logic Optimization:                  0.03
Mapping Optimization:                0.07
----------------------------------------
Overall Compile Time:                0.74
Overall Compile Wall Clock Time:     0.99

--------------------------------------------------------------------
```

Power report

```
Global Operating Voltage = 0.78
Power-specific unit information :
    Voltage Units = 1V
    Capacitance Units = 1.000000ff
    Time Units = 1ns
    Dynamic Power Units = 1uW    (derived from V,C,T units)
    Leakage Power Units = 1pW


Attributes
----------
i - Including register clock pin internal power


  Cell Internal Power  =   4.0581 uW   (95%)
  Net Switching Power  = 213.2434 nW    (5%)
                         ---------
Total Dynamic Power    =   4.2713 uW  (100%)

Cell Leakage Power     = 292.1468 nW


                 Internal         Switching           Leakage            Total
Power Group      Power            Power               Power              Power   (   %    )  Attrs
--------------------------------------------------------------------------------------------------
io_pad             0.0000           0.0000            0.0000            0.0000  (   0.00%)
memory             0.0000           0.0000            0.0000            0.0000  (   0.00%)
black_box          0.0000           0.0000            0.0000            0.0000  (   0.00%)
clock_network      3.3612           0.0000            0.0000            3.3612  (  73.66%)  i
register           0.3876        8.9102e-02        1.6083e+05           0.6376  (  13.97%)
sequential         0.0000           0.0000            0.0000            0.0000  (   0.00%)
combinational      0.3092           0.1241        1.3132e+05            0.5646  (  12.37%)
--------------------------------------------------------------------------------------------------
Total              4.0581 uW        0.2132 uW     2.9215e+05 pW         4.5634 uW
1
dc_shell>
```

Report area

```
****************************************
Report : area
Design : pwm_8bit
Version: V-2023.12-SP4
Date   : Sat Feb 14 16:40:03 2026
****************************************

Library(s) Used:

    saed32rvt_tt0p78vn40c (File: /ve/ws_home/ws_50/RTL2GDSII_TESTCASE/ref/lib/stdcell_rvt/saed32rvt_tt0p78vn40c.db)

Number of ports:                          27
Number of nets:                           77
Number of cells:                          39
Number of combinational cells:            29
Number of sequential cells:                9
Number of macros/black boxes:              0
Number of buf/inv:                         5
Number of references:                     14

Combinational area:                69.635457
Buf/Inv area:                       6.607744
Noncombinational area:             64.044289
Macro/Black Box area:               0.000000
Net Interconnect area:              9.359675

Total cell area:                  133.679746
Total area:                       143.039421
1
dc_shell>
```

FLOORPLAN USING ICCII

Floorplanning is an important stage in the physical design flow where the layout structure of the 8-bit PWM module is defined within the chip area. In this step, the core area, standard cell rows, and I/O pin locations are established before placement begins. Proper floorplanning helps in achieving better area utilization, minimizing routing congestion, and improving overall timing performance.

Since the PWM design mainly consists of standard cells such as flip-flops, counters, and comparator logic, the floorplan is created by defining suitable core utilization and core offsets. Using IC Compiler II (ICCII), the core area is initialized and I/O pins are placed around the boundary of the design. A well-planned floorplan ensures smooth placement and routing in later stages and supports efficient timing closure.

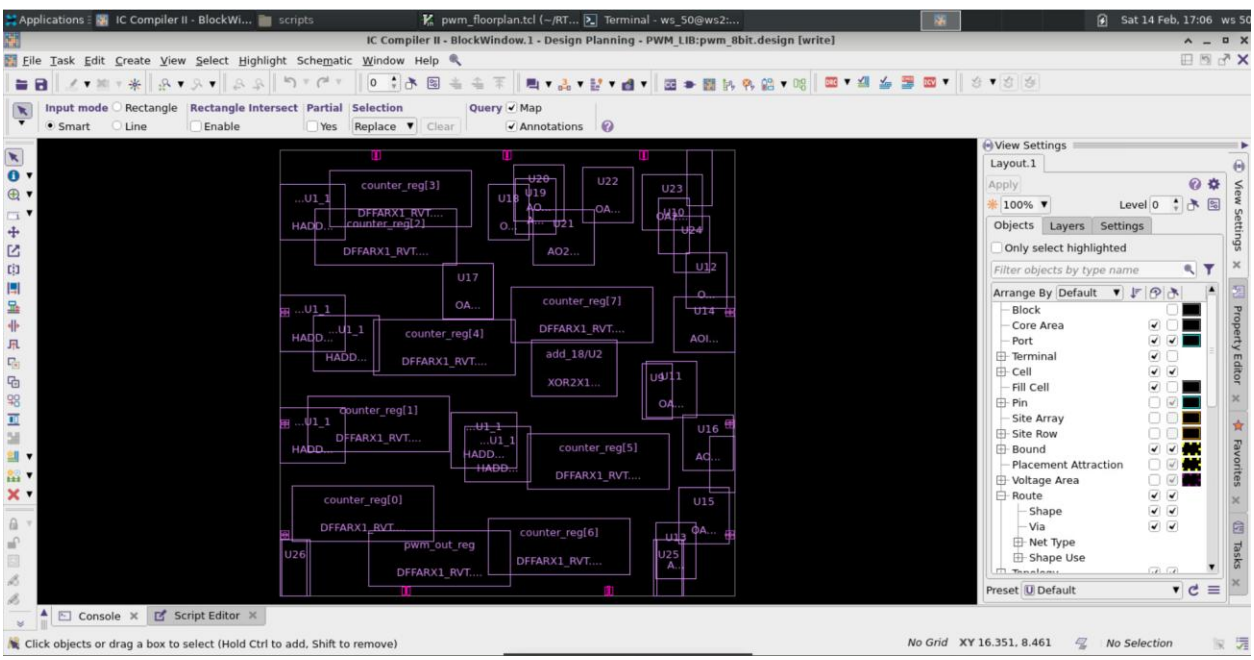
Floorplan.tcl file script

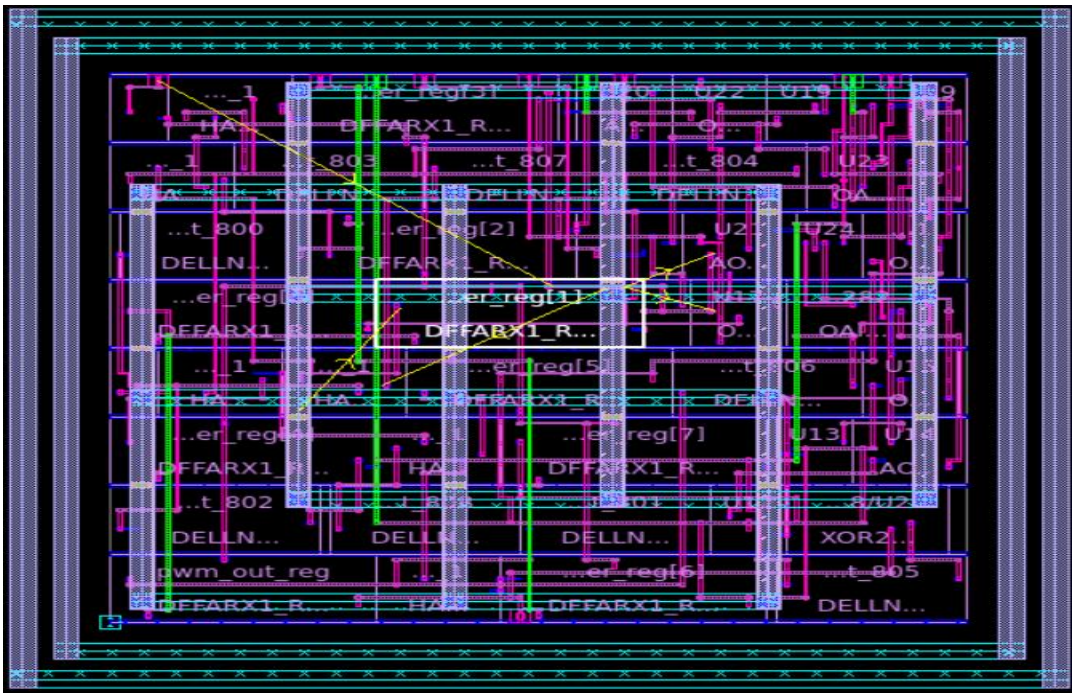
```
set PDK_PATH ./../ref

create_lib -ref_lib $PDK_PATH/lib/ndm/saed32rvt_c.ndm PWM_LIB

read_verilog {./../DC/results/pwm.mapped.v} -library PWM_LIB -design pwm_8bit
-top pwm_8bit

# FloorPlan settings
#######################################################################
####################
#scenario1:
##
initialize_floorplan
place_pins -self
```

```
create_placement -floorplan
#
#save_block
#or
#save_block -as FULL_ADD

#save_lib

#close_block
#close_lib

#open the lib and block using the following two commands:-
#1.open_lib FULL_ADDER_LIB
#2.open_block FULL_ADD

#######################################################################
#################
#
##scenario2:
####
initialize_floorplan -core_offset 2
set_individual_pin_constraints -ports [get_ports] -sides 2
place_pins -self
create_placement -floorplan -effort very_low
##
#
##scenario3:
initialize_floorplan -shape L -core_offset 2 -coincident_boundary true
set_individual_pin_constraints -ports [get_ports {A}] -sides 6
place_pins -self
create_placement -floorplan  -effort medium
#
```

##scenario4:
initialize_floorplan -core_utilization 0.6 -core_offset {3 3} -coincident_boundary false
set_individual_pin_constraints -ports [get_ports {A B}] -sides 2 -pin_spacing_distance 3
place_pins -self
create_placement -floorplan -effort high
#
##End of Floorplan Scenarios#################################

Obtained Floorplan





POWER PLANNING

Power planning is a key step in the physical design flow where a proper power distribution network (PDN) is created to supply stable VDD and VSS across the 8-bit PWM design. Even though the PWM module has moderate power requirements, a well-structured power network is necessary to ensure reliable operation.

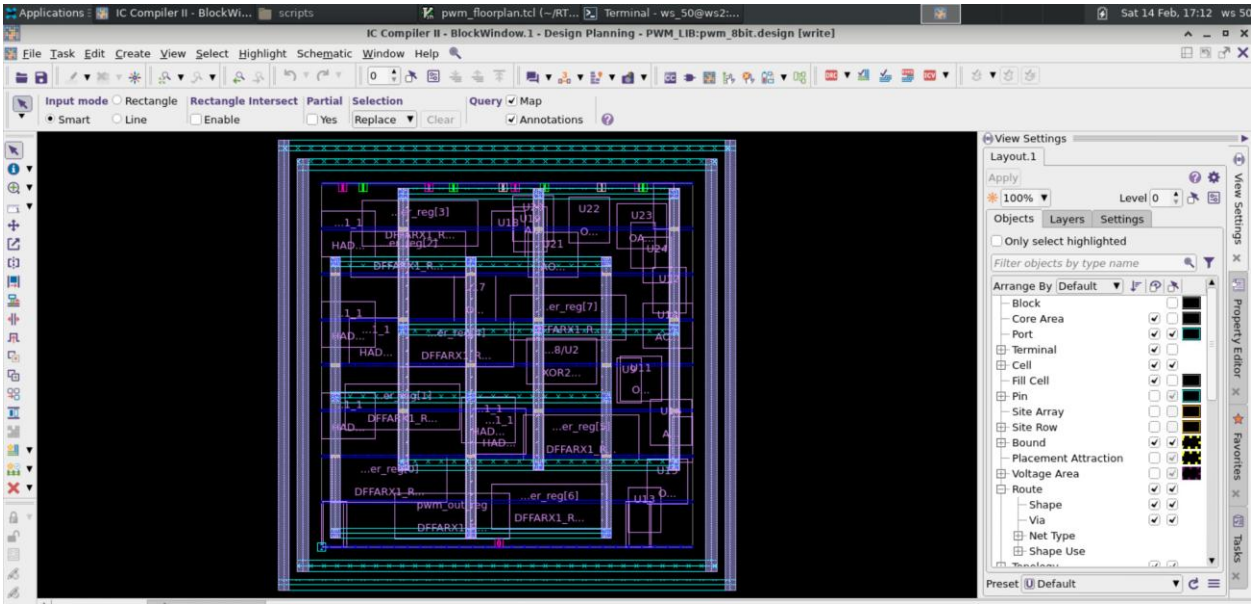
In ICCII, power rings and power connections are defined around the core to distribute power uniformly to all standard cells. Proper power planning helps prevent IR drop issues and supports stable performance during placement, clock tree synthesis, and routing.

Power_plan.tcl file script

```
#################################################################
####################
## Power-planning - To build Power Delivery Network (PDN)
##
#################################################################
##################
#
#################################################################
#########
### Step 1: to_create power/ground nets and to_connect power/ground nets :-
#################################################################
###########

#to create power nets

create_net -power {VDD}
create_net -ground {VSS}

## to connect power/ground_nets
#
connect_pg_net -all_blocks -automatic

#################################################################
##############
### step 2: to create power and ground ring patterns
#################################################################
##############
###
#scenario1:
```

```
create_pg_ring_pattern core_ring_pattern -horizontal_layer M7 -horizontal_width
.4 -horizontal_spacing .3 -vertical_layer M8 -vertical_width .4 -vertical_spacing .3

set_pg_strategy core_power_ring -core -pattern {{name : core_ring_pattern}{nets
: {VDD VSS}}{offset : {.5 .5}}}

compile_pg -strategies core_power_ring

#######################################################################
############
### step 3: to create pg mesh pattern
#######################################################################
#############


create_pg_mesh_pattern mesh -layers { {{vertical_layer: M6}{width: .34} {spacing:
interleaving}{pitch: 5} {offset: .5}} {{horizontal_layer: M7}{width: .38} {spacing:
interleaving} {pitch: 5} {offset: .5}} {{vertical_layer: M8}{width: .38} {spacing:
interleaving}{pitch: 5} {offset: .5}}}

set_pg_strategy core_mesh -pattern { {pattern:mesh} {nets: VDD VSS}} -core -
extension {stop: innermost_ring}

compile_pg -strategies core_mesh


#######################################################
###step 4 : to create std cell power rail pattern
#######################################################

create_pg_std_cell_conn_pattern std_cell_rail -layers {M1} -rail_width 0.09
```

```
set_pg_strategy rail_strat -core -pattern {{name: std_cell_rail} {nets: VDD VSS} }

compile_pg -strategies rail_strat

##############################################################
### step 5 : To save block and Save lib
##############################################################
## "save_block"  will save block as the deign name (full_adder.design) by default
#
save_block

## "save_block -as <preferred name>" to save the block in user preferred name
##
## "save_lib" will save library as created already

save_lib
```



PLACEMENT

Placement is a crucial stage in the physical design flow where standard cells such as flip-flops and combinational logic are positioned within the defined core area after floorplanning. The goal of placement is to optimize area utilization, minimize wirelength, and improve timing performance while satisfying design constraints. For the 8-bit PWM design, placement arranges the counter and comparator logic efficiently to ensure better routing feasibility and support timing closure in later stages.

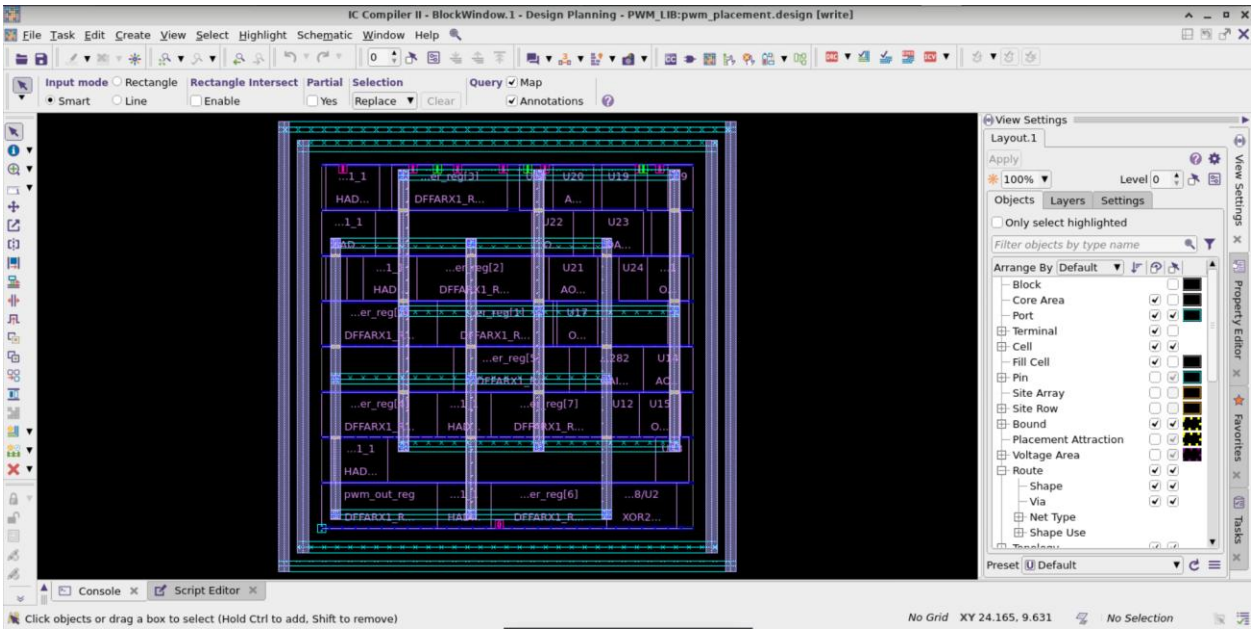
Placement.tcl file script

```
##check for current mode,corner,scneario###
current_mode
current_corner
current_scenario

####mode for placement

remove_modes -all; remove_corners -all; remove_scenarios -all

set mode1 "func"
set corner1 "nom"
set scenario1 "${mode1}::${corner1}"

create_mode $mode1
create_corner $corner1
create_scenario -name func::nom -mode func -corner nom

source ./../CONSTRAINTS/pwm.sdc

#set_dont_use [get_lib_cells */FADD*]
#set_dont_use [get_lib_cells */HADD*]
#set_dont_use [get_lib_cells */AO*]
#set_dont_use [get_lib_cells */OA*]
```

```
#set_dont_use [get_lib_cells */NAND*]
#set_dont_use [get_lib_cells */XOR*]
#set_dont_use [get_lib_cells */NOR*]
#set_dont_use [get_lib_cells */XNOR*]
#set_dont_use [get_lib_cells */MUX*]

current_mode func
current_corner nom
current_scenario func::nom

set parasitic1 "p1"
set tluplus_file$parasitic1
"$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmax.tluplus"
set layer_map_file$parasitic1
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

set parasitic2 "p2"
set tluplus_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_1p9m_Cmin.tluplus"
set layer_map_file$parasitic2
"$PDK_PATH/tech/star_rcxt/saed32nm_tf_itf_tluplus.map"

read_parasitic_tech -tlup $tluplus_filep1 -layermap $layer_map_filep1 -name p1
read_parasitic_tech -tlup $tluplus_filep2 -layermap $layer_map_filep2 -name p2

set_parasitic_parameters -late_spec $parasitic1 -early_spec $parasitic2
set_app_options -name place.coarse.continue_on_missing_scandef -value true

place_pins -self
place_opt

save_block -as pwm_placement
```

save_lib

Placement Schematic:-



CLOCK TREE SYNTHESIS

Clock Tree Synthesis (CTS) is an important stage in the physical design flow where the clock distribution network is constructed to deliver the clock signal to all sequential elements with minimal skew and delay.

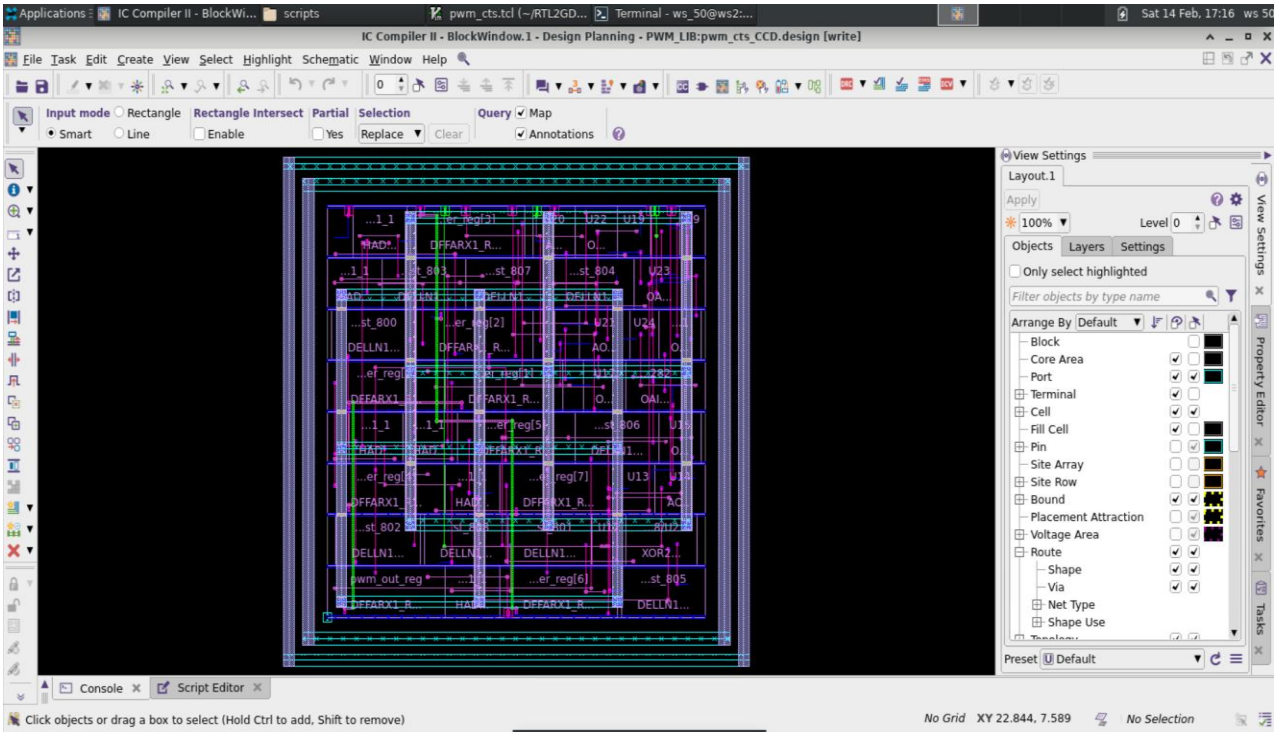
In the 8-bit PWM design, CTS ensures that all flip-flops in the counter and control logic receive a balanced clock signal, helping to maintain proper timing and reliable synchronous operation.

Clock_tree.tcl file script

```
#####################################################
##      CTS using CCD  commands
```

```
#####################################################
##
##stage 1:
##

synthesize_clock_tree
##stage 2:
#
set_app_options -name cts.optimize.enable_local_skew -value true
set_app_options -name cts.compile.enable_local_skew -value true
set_app_options -name cts.compile.enable_global_route -value false
set_app_options -name clock_opt.flow.enable_ccd -value true

#stage 3:
###
#
clock_opt

##############################################################
###  to report qor the respective CTS
###############################################################
#
#report_clock_qor

#############################################################
#  to save the block
#############################################################

save_block -as pwm_cts_CCD
save_lib
```

Schematics:-



ROUTING

Routing is the stage in the physical design flow where metal interconnections are created between the placed standard cells and I/O pins according to the synthesized netlist. This step establishes all signal and power connections required for correct circuit operation.

In the 8-bit PWM design, routing connects the counter, comparator logic, clock network, and power rails while ensuring that timing and design rules are satisfied.

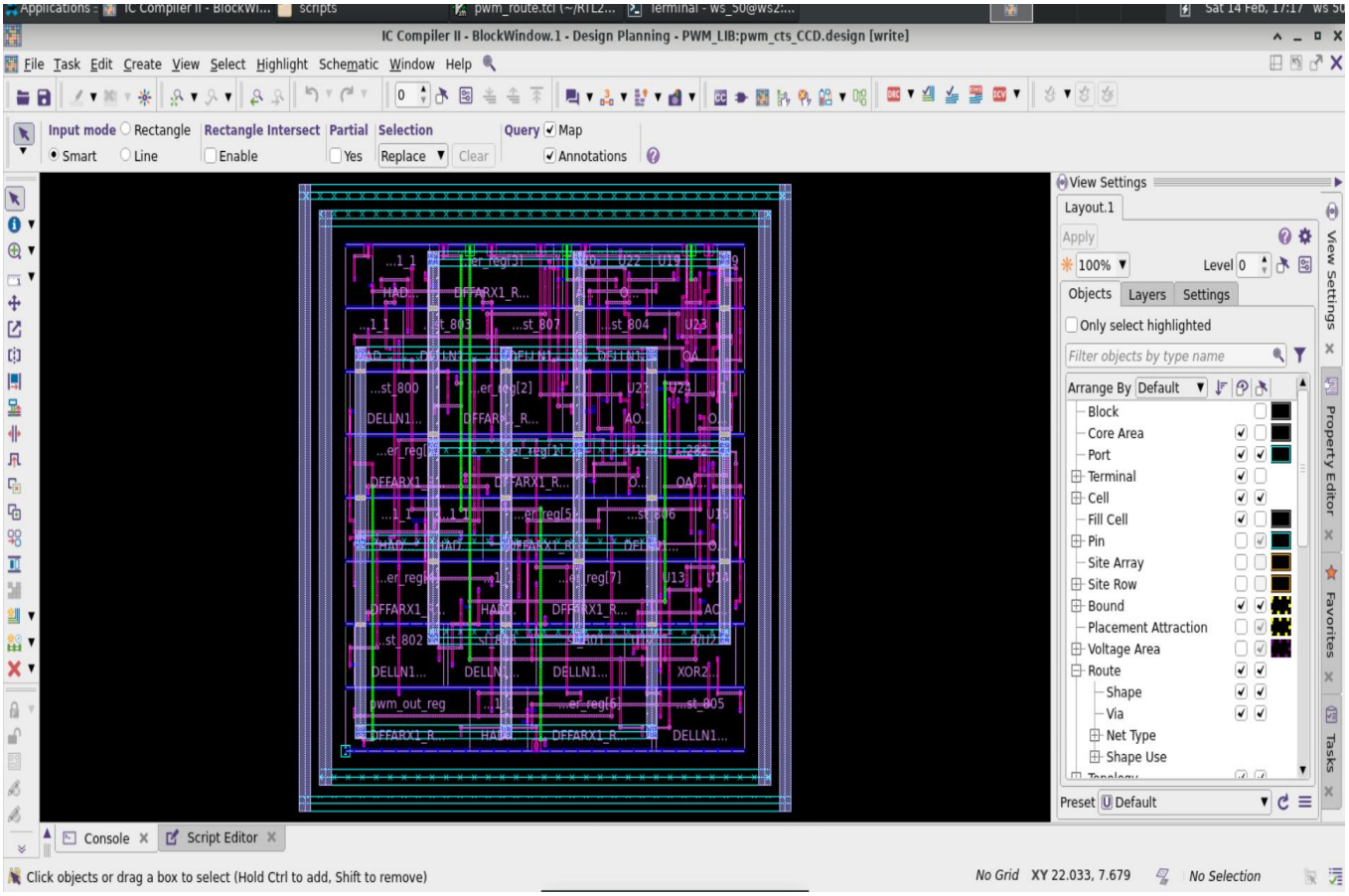
route.tcl file script

```
##########################################################
# set app options
##########################################################
```

```
#
#global route
set_app_options -name route.global.timing_driven -value true
set_app_options -name route.global.crosstalk_driven -value false


#track assignment
set_app_options -name route.track.timing_driven -value true
set_app_options -name route.track.crosstalk_driven -value true
#

#detail route
set_app_options -name route.detail.timing_driven -value true
#set_app_options -name route.detail.save_after_iterations -value false
set_app_options -name route.detail.force_max_number_iterations -value false
set_app_options -name route.detail.antenna -value true
set_app_options -name route.detail.antenna_fixing_preference -value use_diodes
set_app_options -name route.detail.diode_libcell_names -value */ANTENNA_RVT
#

####################################
#Atomic commands for route_auto
####################################
route_global
#save_block route_global_database

route_track
#save_block route_track_database

route_detail
#save_block route_detail_database
```

```
route_auto



route_opt

#######################################################
#to add filler cells
#######################################################
#
#set fillers_ref "*/*SHFILL128_RVT */*SHFILL64_RVT */*SHFILL3_RVT
*/*SHFILL2_RVT */*SHFILL1_RVT"
#create_stdcell_fillers -lib_cell $fillers_ref

#######################################################
# OUTPUTS
#######################################################
write_verilog ./results/pwm.routed.v
write_sdc -output ./results/pwm.routed.sdc
write_parasitics -format spef -output ./results/pwm_${scenario1}.spef

#########################################################
# To write GDSII
#########################################################

write_gds -merge_files " $PDK_PATH/lib/gds/saed32nm_rvt_oa.gds" pwm.gds
#
```

Schematic after Routing



QOR Report

```
****************************************
Report : qor
Design : pwm_8bit
Version: V-2023.12-SP4
Date   : Sat Feb 14 17:18:29 2026
****************************************


Scenario           'func::nom'
Timing Path Group  '**in2reg_default**'
----------------------------------------
Levels of Logic:                      8
Critical Path Length:              2.33
Critical Path Slack:               7.44
Critical Path Clk Period:         10.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------

Scenario           'func::nom'
Timing Path Group  '**reg2out_default**'
----------------------------------------
Levels of Logic:                      0
Critical Path Length:              0.08
Critical Path Slack:               7.72
Critical Path Clk Period:         10.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

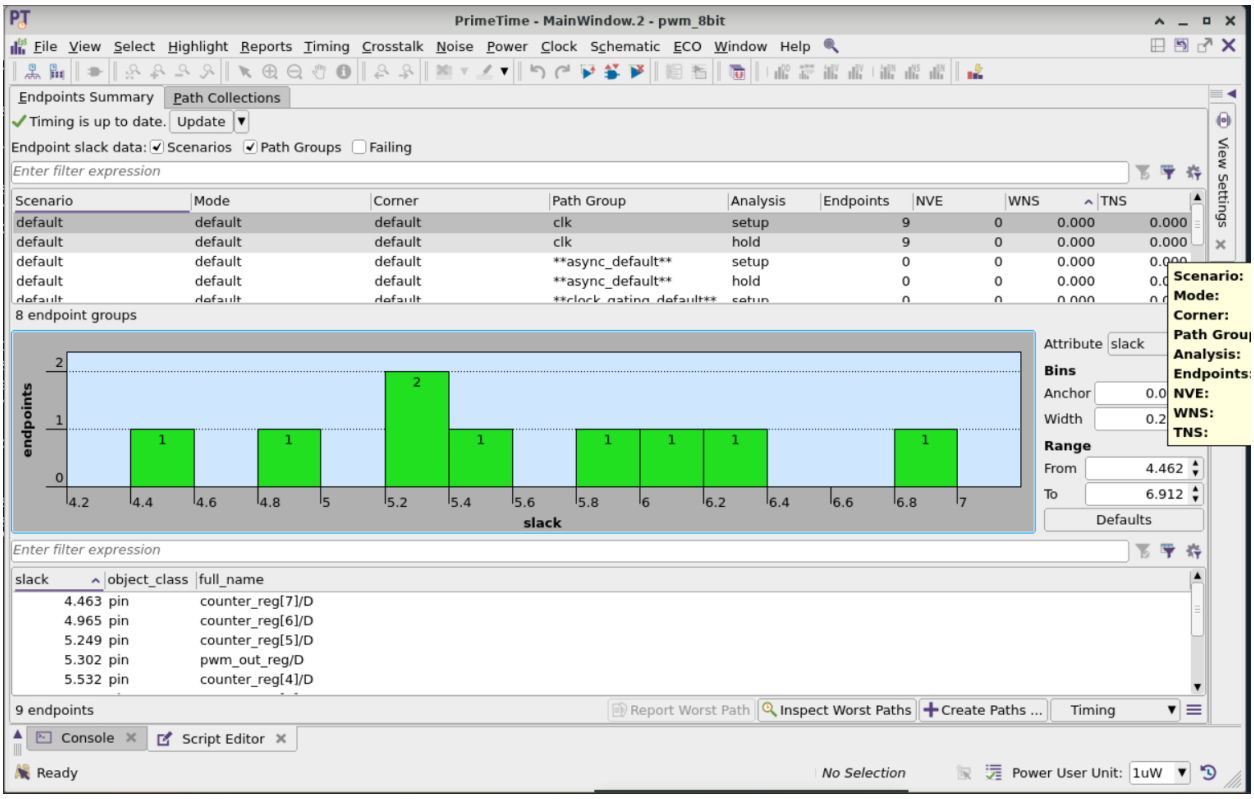
```
Scenario            'func::nom'
Timing Path Group   'clk'
----------------------------------------
Levels of Logic:                       8
Critical Path Length:               0.47
Critical Path Slack:                9.30
Critical Path Clk Period:          10.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------
```

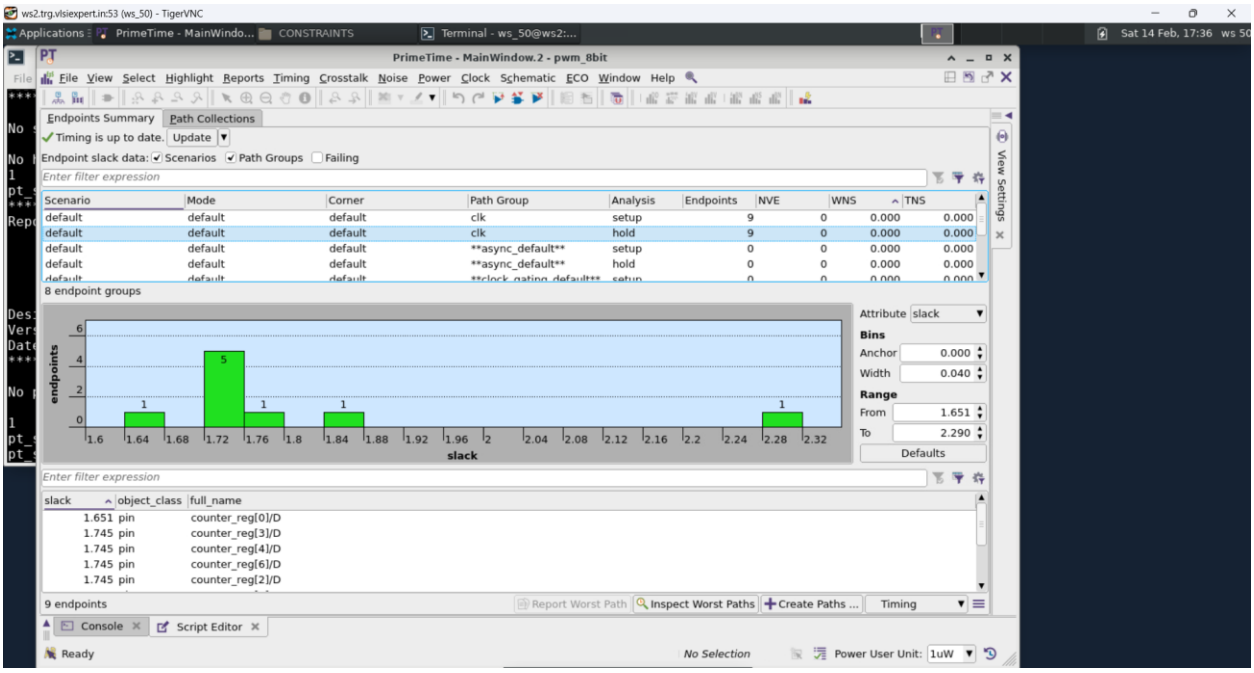
```
Cell Count
----------------------------------------
Hierarchical Cell Count:              1
Hierarchical Port Count:             18
Leaf Cell Count:                     47
Buf/Inv Cell Count:                  15
Buf Cell Count:                       9
Inv Cell Count:                       6
Combinational Cell Count:            38
   Single-bit Isolation Cell Count:                        0
   Multi-bit Isolation Cell Count:                         0
   Isolation Cell Banking Ratio:                           0.00%
   Single-bit Level Shifter Cell Count:                    0
   Multi-bit Level Shifter Cell Count:                     0
   Level Shifter Cell Banking Ratio:                       0.00%
   Single-bit ELS Cell Count:                              0
   Multi-bit ELS Cell Count:                               0
   ELS Cell Banking Ratio:                                 0.00%
Sequential Cell Count:                9
   Integrated Clock-Gating Cell Count:                     0
   Sequential Macro Cell Count:                            0
   Single-bit Sequential Cell Count:                       9
   Multi-bit Sequential Cell Count:                        0
   Sequential Cell Banking Ratio:                          0.00%
   BitsPerflop:                                            1.00
Macro Count:                          0
----------------------------------------
```

```
Area
----------------------------------------
Combinational Area:                115.89
Noncombinational Area:              64.04
Buf/Inv Area:                       53.62
Total Buffer Area:                  45.75
Total Inverter Area:                 7.88
Macro/Black Box Area:                0.00
Net Area:                               0
Net XLength:                       190.46
Net YLength:                       266.81
----------------------------------------
Cell Area (netlist):                            179.93
Cell Area (netlist and physical only):          179.93
Net Length:                        457.27


Design Rules
----------------------------------------
Total Number of Nets:                  72
Nets with Violations:                   0
Max Trans Violations:                   0
Max Cap Violations:                     0
----------------------------------------

1
icc2_shell>
```

STA USING PRIME TIME

Static Timing Analysis (STA) is a crucial step in the digital VLSI design flow used to validate the timing performance of a circuit without applying any test vectors. It checks whether the design meets timing constraints such as setup and hold times for every path between flipflops and primary inputs/outputs.




ws2.trg.vlsiexpert.in:53 (ws_50) - TigerVNC
Applications
PrimeTime - MainWindo...
CONSTRAINTS
Terminal - ws_50@ws2:...
Sat 14 Feb, 17:36
PrimeTime - MainWindow.2 - pwm_8bit
File View Select Highlight Reports Timing Crosstalk Noise Power Clock Schematic ECO Window Help
Endpoints Summary
Path Collections
Timing is up to date.
Update
Endpoint slack data: Scenarios Path Groups Failing
Enter filter expression
Scenario Mode Corner Path Group Analysis Endpoints NVE WNS TNS
default default default clk setup 9 0 0.000 0.000
default default default clk hold 9 0 0.000 0.000
default default default **async_default** setup 0 0 0.000 0.000
default default default **async_default** hold 0 0 0.000 0.000
8 endpoint groups
endpoints
slack
Attribute slack
Bins
Anchor 0.000
Width 0.040
Range
From 1.651
To 2.290
Defaults
slack object_class full_name
1.651 pin counter_reg[0]/D
1.745 pin counter_reg[3]/D
1.745 pin counter_reg[4]/D
1.745 pin counter_reg[6]/D
1.745 pin counter_reg[2]/D
9 endpoints
Report Worst Path
Inspect Worst Paths
Create Paths ...
Timing
Console
Script Editor
Ready
No Selection
Power User Unit: 1uW
View Settings

Report qor

```
****************************************
Report : qor
Design : pwm_8bit
Version: W-2024.09-SP1
Date   : Sat Feb 14 23:27:10 2026
****************************************

  Timing Path Group 'clk' (max_delay/setup)
  -----------------------------------------
  Levels of Logic:                         9
  Critical Path Length:                 4.68
  Critical Path Slack:                  4.46
  Total Negative Slack:                 0.00
  No. of Violating Paths:                  0
  -----------------------------------------

  Timing Path Group 'clk' (min_delay/hold)
  ----------------------------------------
  Levels of Logic:                         3
  Critical Path Length:                 1.80
  Critical Path Slack:                  1.65
  Total Negative Slack:                 0.00
  No. of Violating Paths:                  0
  ----------------------------------------

  Area
  ----------------------------------------
  Net Interconnect area:               10.10
  Total cell area:                    179.93
  Design Area:                        190.04
  ----------------------------------------
```

```
Cell & Pin Count
---------------------------------------------
Pin Count:                                169
Hierarchical Cell Count:                    1
Hierarchical Port Count:                   16
Leaf Cell Count:                           47
---------------------------------------------


Design Rule Violations
---------------------------------------------
Total No. of Pins in Design:              169
Total DRC Cost:                          0.00
---------------------------------------------
```

Timing report

```
Path Type: max

Point                                                    Incr       Path
-----------------------------------------------------------------------------
clock clk (rise edge)                                    0.00       0.00
clock network delay (ideal)                              0.00       0.00
counter_reg[1]/CLK (DFFARX1_RVT)                         0.00       0.00 r
counter_reg[1]/Q (DFFARX1_RVT)                           0.85       0.85 r
add_18/copt_h_inst_807/Y (DELLN1X2_RVT)                  1.39       2.24 r
add_18/G1/G1_1(1)/U1_1/C1 (HADDX1_RVT)                   0.30       2.53 r
add_18/G1/G1_1(2)/U1_1/C1 (HADDX1_RVT)                   0.28       2.82 r
add_18/G1/G1_1(3)/U1_1/C1 (HADDX1_RVT)                   0.28       3.10 r
add_18/G1/G1_1(4)/U1_1/C1 (HADDX1_RVT)                   0.28       3.38 r
add_18/G1/G1_1(5)/U1_1/C1 (HADDX1_RVT)                   0.28       3.67 r
add_18/G1/G1_1(6)/U1_1/C1 (HADDX1_RVT)                   0.28       3.95 r
add_18/U2/Y (XOR2X1_RVT)                                 0.73       4.68 f
counter_reg[7]/D (DFFARX1_RVT)                           0.00       4.68 f
data arrival time                                                   4.68

clock clk (rise edge)                                   10.00      10.00
clock network delay (ideal)                              0.00      10.00
counter_reg[7]/CLK (DFFARX1_RVT)                                   10.00 r
clock reconvergence pessimism                            0.00      10.00
clock uncertainty                                       -0.20       9.80
library setup time                                      -0.65       9.15
data required time                                                  9.15
-----------------------------------------------------------------------------
data required time                                                  9.15
data arrival time                                                  -4.68
-----------------------------------------------------------------------------
slack (MET)                                                         4.46


1
pt_shell>
```

CONCLUSION

The design and implementation of the 8-bit Pulse Width Modulation (PWM) module were successfully carried out using Verilog HDL and Synopsys EDA tools. The project covered all major stages of the ASIC design flow, including RTL development, functional simulation, logic synthesis, physical design using ICC2, and static timing analysis using PrimeTime. Each stage was carefully verified to ensure that the final design was functionally accurate, area-efficient, and met the required timing constraints.

This project enhanced the understanding of synchronous digital design concepts such as counters, comparators, and clock-driven logic, while also providing valuable hands-on experience with industry-standard VLSI tools. It served as a practical platform to bridge theoretical knowledge with real-world ASIC implementation, laying a strong foundation for advanced digital and physical design projects.